Lecture: Bipartite Graph Check — From Zero to Advanced

1. Introduction

A **bipartite graph** is a graph whose vertices can be **divided into two disjoint sets**, **U** and **V**, such that every edge connects a vertex in **U** to one in **V**.
No edge connects vertices **within the same set**.

2. Key Idea

The **Bipartite Check** verifies if a graph can be **colored using two colors** (say, Red and Blue) such that **no two adjacent vertices have the same color**.

If such a coloring is possible → the graph is **bipartite** ✅
If not (i.e., an odd-length cycle exists) → the graph is **not bipartite** ❌

3. Real-Life Analogy

Imagine dividing a group of people into **two teams** where:

- Every friendship link (edge) must connect **members from different teams**.
- No friendship can exist **within the same team**.

If this can be done → the network is **bipartite**.

4. When to Use Bipartite Check

Use Case	Example
Scheduling	Split tasks into two sets where no dependent tasks overlap
Team Assignment	Divide workers to avoid conflicts
Graph Coloring	Check 2-colorability
Cycle Property	Detect odd-length cycles

5. Properties of a Bipartite Graph

Property	Description
No odd-length cycles	A bipartite graph cannot have odd cycles
2-colorable	Vertices can be colored with two colors

Property	Description
Can use BFS/DFS	To check bipartiteness efficiently
Works on	Directed/Undirected graphs (commonly undirected)

6. Algorithm Intuition

We can check bipartiteness using **BFS or DFS** by assigning colors alternately:

- Assign color 0 to a starting node.
- Assign opposite color 1 to all its neighbors.
- Continue until all nodes are visited.
- If any two adjacent nodes share the same color → ❌ not bipartite.

7. Steps (BFS-Based Approach)

1. Initialize all nodes with color -1 (uncolored).
2. Start from any vertex → color it 0.
3. Perform BFS traversal:
    - For every neighbor:
        - If **uncolored**, assign **opposite color**.
        - If **already same color**, graph is **not bipartite**.
4. Repeat for all components (if disconnected).

8. Example Graph

Graph Representation

```
1 —— 2
|   /
|  /
3 —— 4
```

✅ This graph **is bipartite**.
Possible division:

- Set 1 = {1, 4}
- Set 2 = {2, 3}

Counter Example (Not Bipartite)

```
 1 —— 2
 \   /
  \ /
   3
```

❌ This triangle has an **odd-length cycle**, so it's **not bipartite**.

---

9. Pseudocode (BFS Method)

```
from collections import deque


def is_bipartite(graph):
    color = {}
    for node in graph:
        if node not in color:
            queue = deque([node])
            color[node] = 0

            while queue:
                curr = queue.popleft()
                for neighbor in graph[curr]:
                    if neighbor not in color:
                        color[neighbor] = 1 - color[curr]
```

```
queue.append(neighbor)
            elif color[neighbor] == color[curr]:
                return False  # Same color conflict
    return True
```

✅ Works for connected and disconnected graphs
✅ Uses BFS traversal to alternate colors

10. Example Execution

Input:

```
graph = {
    1: [2, 3],
    2: [1, 4],
    3: [1, 4],
    4: [2, 3]
}

print(is_bipartite(graph))
```

Output:

```
True
```

✅ Graph is bipartite → can be divided into {1, 4} and {2, 3}.

11. DFS-Based Alternative

```
ef dfs_bipartite(graph, node, color, curr_color):
    color[node] = curr_color
    for neighbor in graph[node]:
        if neighbor not in color:
            if not dfs_bipartite(graph, neighbor, color, 1 - curr_color):
```

```
return False
        elif color[neighbor] == color[node]:
            return False
    return True



def is_bipartite_dfs(graph):
    color = {}
    for node in graph:
        if node not in color:
            if not dfs_bipartite(graph, node, color, 0):
                return False
    return True
```

✅ Recursive approach
✅ Same time complexity as BFS

12. Complexity Analysis

Aspect	**Complexity**
Time Complexity	O(V + E)
Space Complexity	O(V)
Algorithm Type	Graph Traversal (BFS/DFS)
Works For	Directed & Undirected Graphs

13. Edge Cases

Case	**Behavior**
Disconnected Graph	Check each component separately

Case	Behavior
Self-loop	❌ Not bipartite
Empty Graph	✅ Trivially bipartite
Single Edge	✅ Always bipartite

14. Real-World Applications

Domain	Application
Scheduling	Separate dependent tasks
Social Networks	Group people with no internal connections
AI & ML	Clustering problems
Database Systems	Dependency graph verification
Conflict Resolution	Assign non-conflicting teams/groups

15. Summary Table

Feature	Bipartite Graph Check
Type	Graph Coloring
Graph Type	Undirected / Directed
Approach	BFS or DFS
Detects	Odd cycles (non-bipartite)
Time Complexity	O(V + E)
Space Complexity	O(V)
Applications	Scheduling, Grouping, Networks

16. Practice Tasks

1. Implement **BFS-based bipartite check**.

2. Implement **DFS-based version** and compare.
3. Test on **connected** and **disconnected** graphs.
4. Add **cycle detection** to verify non-bipartite graphs.
5. Visualize color assignment for each vertex.

17. Closing Thought

The **Bipartite Graph Check** is a fundamental concept that connects **graph coloring**, **cycle detection**, and **partitioning**.

It's simple, elegant, and widely used in problems involving **conflicts, dependencies, and compatibility**.

Once you understand bipartite graphs, you're ready for advanced topics like **graph matching** and **flow networks** 🚀